Grand Prix of Wrocław

E: All in good fun!

Given three lines we have to find the point, which minimalize the maximum distance to given lines and output this distance.

Fact

We know that incenter is the point equidistant from the triangle's sides, so we only have to find the required distance, which is the radius of the inscribed circle of the triangle.

Fact

We know that incenter is the point equidistant from the triangle's sides, so we only have to find the required distance, which is the radius of the inscribed circle of the triangle.

Formula

$$r = \frac{2 \cdot \Delta}{P}$$

r - radius of the inscribed circle
Δ - area of the triangle, can be computed by cross product
P - perimeter of the triangle

Fact

We know that incenter is the point equidistant from the triangle's sides, so we only have to find the required distance, which is the radius of the inscribed circle of the triangle.

Formula

$$r = \frac{2 \cdot \Delta}{P}$$

r - radius of the inscribed circle
Δ - area of the triangle, can be computed by cross product
P - perimeter of the triangle

Challenge

What if you are given a huge set of lines (for example $N \leq 50\,000$) and need to find three of them for which radius is maximized?

B: The Witcher

You are given a graph where each edge is either black or white. For each edge you know if you can change its color. Your goal is to change colors of some edges so each vertex has even number of adjacent black edges.

Let's say that the vertex v is *black* if the number of adjacent edges (with one endpoint in v) with $x_i = 1$ is odd and *white* if this number is even. Our task is to find such a subset of edges with $x_i = 0$, that each black vertex has odd number of adjacent edges and each white vertex has even number of adjacent edges.

Let's say that the vertex v is *black* if the number of adjacent edges (with one endpoint in v) with $x_i = 1$ is odd and *white* if this number is even. Our task is to find such a subset of edges with $x_i = 0$, that each black vertex has odd number of adjacent edges and each white vertex has even number of adjacent edges.

So now we may forget about edges with $x_i = 1$ and just remember the colours of vertices. Note that now the graph doesn't have to be connected. We know that in each connected component the sum of all degrees must be even, so the number of black vertices must be even. It turns out that this condition is sufficient and necessary.

Let's say that the vertex v is *black* if the number of adjacent edges (with one endpoint in v) with $x_i = 1$ is odd and *white* if this number is even. Our task is to find such a subset of edges with $x_i = 0$, that each black vertex has odd number of adjacent edges and each white vertex has even number of adjacent edges.

So now we may forget about edges with $x_i = 1$ and just remember the colours of vertices. Note that now the graph doesn't have to be connected. We know that in each connected component the sum of all degrees must be even, so the number of black vertices must be even. It turns out that this condition is sufficient and necessary.

To show that this is sufficient we will construct the answer (for each connected component separately). Let's take one of them and take any spanning tree on this vertices. Now let's fix an edge from this tree. This edge naturally divides the vertices into two groups. Now, we will take this edge $\iff$ the number of black vertices in both this groups is odd.

Proof

Consider two cases. Firstly, let's take a white vertex w and root a tree in w. Now the edge going from w to some w_u is taken $\iff$ the subtree of w_u has an odd number of black vertices inside. Because w is white, the total number of black vertices in its subtree is even and the number of subtrees with odd number of black vertices is also even, so the number of taken adjacent edges is also even, which is what we would like to show. The case when a vertex is black is similar, but now we will have an odd number of subtrees with odd number of vertices inside, so the number of taken edges adjacent to this black vertex will be odd.

Proof

Consider two cases. Firstly, let's take a white vertex w and root a tree in w. Now the edge going from w to some w_u is taken $\iff$ the subtree of w_u has an odd number of black vertices inside. Because w is white, the total number of black vertices in its subtree is even and the number of subtrees with odd number of black vertices is also even, so the number of taken adjacent edges is also even, which is what we would like to show. The case when a vertex is black is similar, but now we will have an odd number of subtrees with odd number of vertices inside, so the number of taken edges adjacent to this black vertex will be odd.

To find these taken edges we should root a tree in any vertex and simply run one DFS, which will calculate the number of black vertices in each subtree. The complexity is $\mathcal{O}(n + m)$.

F: Five Nights at Freddy's

We have n types of cameras, each of which has to be included in a cycle of transmission. No two occurrences of the camera i can be further than a_i places apart on the cycle. Determine if it is possible to create such cycle.
Additional constraint: For each two i and j such that $1 \leq i < j \leq n$ at least one of $a_i | a_j$ and $a_j | a_i$ is true.
If it is possible to find such cycle with the length less than or equal to 10^6, output this cycle.

Observation 1

Due to divisibility constraints there can be only logarithmicly many different numbers in the array.

Observation 1

Due to divisibility constraints there can be only logarithmicly many different numbers in the array.

Lemma 2

One can see that if $\sum_1^n \frac{1}{a_i} > 1$, then we cannot construct such cycle.

Observation 1

Due to divisibility constraints there can be only logarithmicly many different numbers in the array.

Lemma 2

One can see that if $\sum_1^n \frac{1}{a_i} > 1$, then we cannot construct such cycle.

Proof

Let's assume that $\sum_1^n \frac{1}{a_i} > 1$, and there exists a solution. Let the final cycle C have length L. At the very minimum i-th camera has to occupy $\frac{L}{a_i}$ positions in C (due to the **Dirichlet's drawer principle**). If we sum those values over all cameras we get that the cycle is too short to pass the constraints, thus we get a contradiction.

Construction

If $\sum_1^n \frac{1}{a_i} \leq 1$, we can show method of constructing cycle C. It's length will be equal to the value of the biggest element in the array, so it will be $\leq 10^6$.

Knowing the length of the cycle we will try to fill it with necessary numbers. It turns out that greedy approach will work. Starting from the cameras with the lowest a_i value, we will place them every a_i positions in the cycle (starting from the first empty spot). It can be shown by induction, that it will always be possible to place first occurrence of i-th camera on the position $\leq a_i$, and all the other positions must not be occupied because of divisibility constraints and the fact that we consider elements in non decreasing order. If after that process, there will remain any empty spots, fill them with arbitrary camera numbers.
Overall complexity: $\mathcal{O}(\max_{1 \leq i \leq n} a_i)$

H: Cheat

You are given a directed graph, with following property – for each $i \in [1, n-1]$ there is an edge from vertex i to $i+1$. Vertex v is good if and only if graph is acyclic after removing v from graph. Find all good vertices.

Observation 1

If there exist two edges $(u_1 \to v_1)$ and $(u_2 \to v_2)$ fulfilling $u_1 > v_1 > u_2 > v_2$ then there are no good vertices.

Observation 1

If there exist two edges $(u_1 \to v_1)$ and $(u_2 \to v_2)$ fulfilling $u_1 > v_1 > u_2 > v_2$ then there are no good vertices.

Observation 2

Assume that there is no such pair of edges as stated in Observation 1. If we draw all edges $u \to v$ fulfilling $u > v$ as arrows from element u to element v then there must exist nonempty interval $[L, R]$ such that all arrows passes above this interval (and we can't take bigger interval). We can notice that only vertices in this interval can be good.

Observation 3

Vertex v is good if and only if it belongs to interval $[L, R]$ and there are no such edges $u_1 \to v_1$, $u_2 \to v_2$ fullfiling $v_2 \leq u_1 < v < v_1 \leq u_2$.

Observation 3

Vertex v is good if and only if it belongs to interval $[L, R]$ and there are no such edges $u_1 \to v_1$, $u_2 \to v_2$ fullfiling $v_2 \leq u_1 < v < v_1 \leq u_2$.

Finding such edges can be done with simple iteration over graph (it is actually better to think that graph is an array with arrows). Final complexity $\mathcal{O}(n + m)$.

Observation 3

Vertex v is good if and only if it belongs to interval $[L, R]$ and there are no such edges $u_1 \to v_1$, $u_2 \to v_2$ fullfiling $v_2 \leq u_1 < v < v_1 \leq u_2$.

Finding such edges can be done with simple iteration over graph (it is actually better to think that graph is an array with arrows). Final complexity $\mathcal{O}(n + m)$.

Challenge

Solve this for arbitrary graph.

L: Boss of all bosses

In this problem we are asked to compute the minimum cost of a permutation of vertices of a weighted tree, where the cost of a permutation is defined as the sum of distances in the tree for all adjacent pairs of elements in this permutation.

Let's focus on what each edge of the tree is contributing to the sum.
It is obvious that every edge will be added at least once.

Lemma

Consider a set of edges that are added exactly once in some solution.
There must exist a **simple** path containing all edges from that set.

The proof is pretty straightforward and it is easy to get the intuition of why it is true by drawing some trees.

Observation

There exists a solution in which every edge is added two times to the sum, except the edges on the heaviest path which are added once.

Indeed, the permutation of that cost is just the pre-order traversal of the tree rooted at one end of the heaviest path, that descends to the vertices lying on the heaviest path after descending to the other neighbors.
It follow from the lemma, that a better solution doesn't exist.

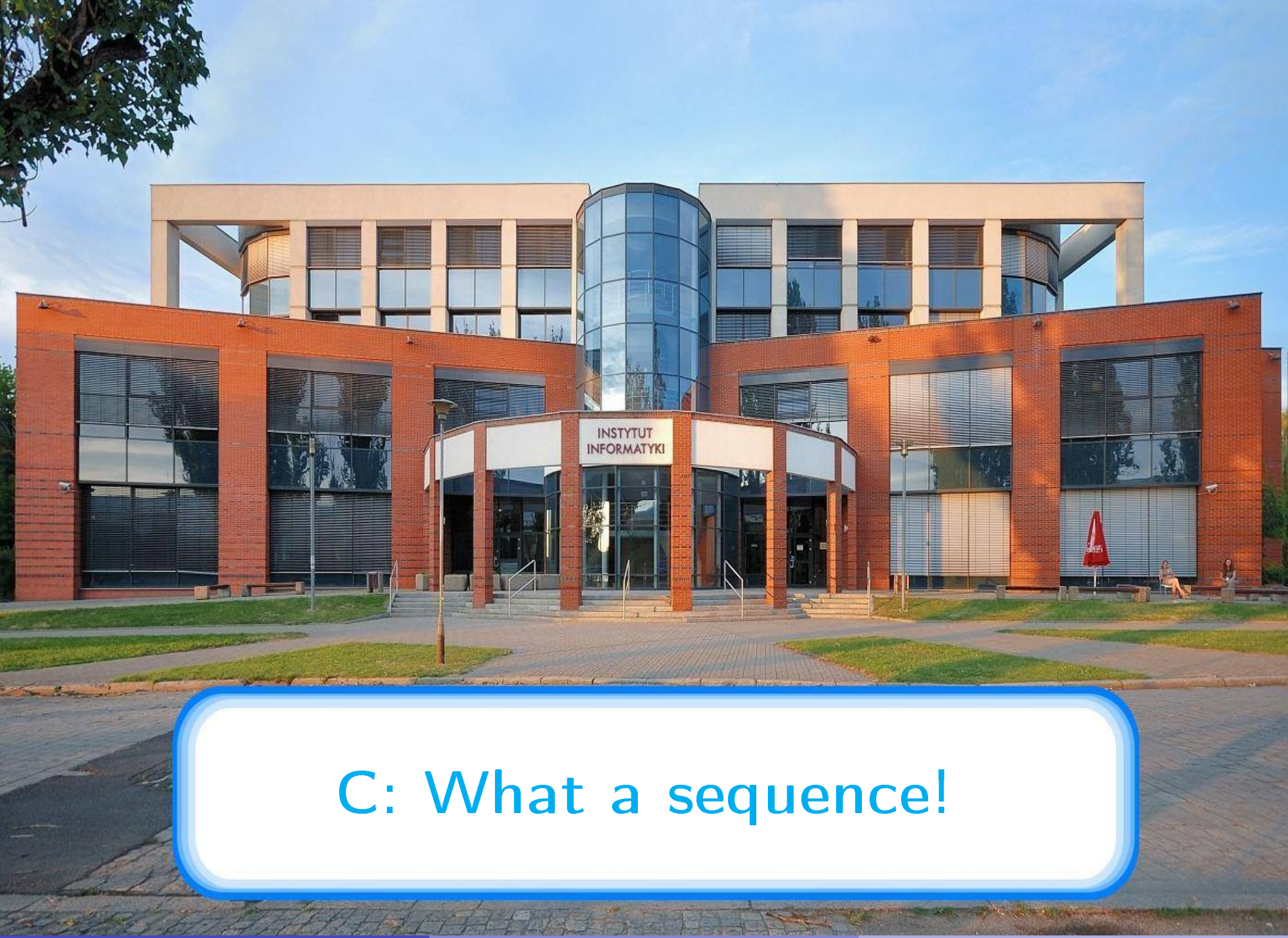
Observation

There exists a solution in which every edge is added two times to the sum, except the edges on the heaviest path which are added once.

Indeed, the permutation of that cost is just the pre-order traversal of the tree rooted at one end of the heaviest path, that descends to the vertices lying on the heaviest path after descending to the other neighbors.
It follow from the lemma, that a better solution doesn't exist.

Challenge

Solve this for cactus.



C: What a sequence!

We are given recursive formula:

$$a_n = k \cdot a_{n-1} + a_{n-2}, a_0 = 0, a_1 = 1$$

where $k \in \{1, 3, 5, 7\}$ and $\log_{10} p \leq 10^6$, p is odd prime number. Our task is to find $a_p \mod p$.

Naive solution

We can solve task in $\mathcal{O}(\log^3 p)$ using matrix exponentation with naive multiplication or $\mathcal{O}(\log^2 p \log\log p)$ using FFT, but that's too slow. We are aiming to solve it in $\mathcal{O}(\log p)$.

Naive solution

We can solve task in $\mathcal{O}(\log^3 p)$ using matrix exponentation with naive multiplication or $\mathcal{O}(\log^2 p \log\log p)$ using FFT, but that's too slow. We are aiming to solve it in $\mathcal{O}(\log p)$.

Formula

We can solve given recurence and obtain formula for a_n:

$$a_n = \frac{1}{\sqrt{k^2+4}} \left(\left(\frac{k+\sqrt{k^2+4}}{2} \right)^n - \left(\frac{k-\sqrt{k^2+4}}{2} \right)^n \right)$$

Naive solution

We can solve task in $\mathcal{O}(\log^3 p)$ using matrix exponentation with naive multiplication or $\mathcal{O}(\log^2 p \log\log p)$ using FFT, but that's too slow. We are aiming to solve it in $\mathcal{O}(\log p)$.

Formula

We can solve given recurence and obtain formula for a_n:

$$a_n = \frac{1}{\sqrt{k^2+4}} \left(\left(\frac{k+\sqrt{k^2+4}}{2} \right)^n - \left(\frac{k-\sqrt{k^2+4}}{2} \right)^n \right)$$

Formula

What can be rewritten for p prime as:

$$a_p = \frac{p \cdot (\dots) + (k^2+4)^{\frac{p-1}{2}}}{2^{p-1}}$$

$$a_p \cdot 2^{p-1} \equiv (k^2+4)^{\frac{p-1}{2}} \pmod{p}$$

Observation

Let's observe that for $k \in \{1, 3, 5, 7\}$ we have $k^2 + 4 \in \{5, 13, 29, 53\}$, $k^2 + 4$ is also a prime, so $a_p \equiv \left(\frac{k^2+4}{p}\right)$ (Legendre symbol).

Observation

Let's observe that for $k \in \{1, 3, 5, 7\}$ we have $k^2 + 4 \in \{5, 13, 29, 53\}$, $k^2 + 4$ is also a prime, so $a_p \equiv \left(\frac{k^2+4}{p}\right)$ (Legendre symbol).

Observation

Also k is odd, so $k^2 + 4 \equiv 1 \pmod 4$, hence from law of quadratic reciprocity:

$$\left(\frac{k^2+4}{p}\right) = \left(\frac{p}{k^2+4}\right)$$

Observation

Let's observe that for $k \in \{1, 3, 5, 7\}$ we have $k^2 + 4 \in \{5, 13, 29, 53\}$, $k^2 + 4$ is also a prime, so $a_p \equiv \left(\frac{k^2+4}{p}\right)$ (Legendre symbol).

Observation

Also k is odd, so $k^2 + 4 \equiv 1 \pmod 4$, hence from law of quadratic reciprocity:

$$\left(\frac{k^2+4}{p}\right) = \left(\frac{p}{k^2+4}\right)$$

Algorithm

We can precompute values of $(\frac{r}{k^2+4})$ for $0 \leq r < k^2 + 4$, cause $(\frac{r}{k^2+4}) = r^{\frac{k^2+4-1}{2}}$ and for each p output answer for $p \mod k^2 + 4$.

Complexity: $\mathcal{O}(\log p)$

G: Choreography

We are given a matrix filled with numbers from $[1, \ldots, n^2]$, such that every number occurs only once. We are asked how many *phases* do we need to make every row and column of the matrix sorted. During one phase we can either permute numbers in each column independently, or in each row.

Zero phases

When the given matrix is already sorted, we don't need any phases.

One phase

We can sort the matrix in one phase if every row/column is a permutation of this row/column in the sorted matrix.

Two phases

Matrix can be sorted in two phases, if either for each row, every number in this row should finally be in a different column, or for each column, every number in this column should finally be in a different row.

It's easy to see, that this condition is sufficient. It turns out that it's also necessary.

Proof

Suppose that the first phase is horizontal. Then we must place every element in the correct column, because we won't be able to change it later. Then in every row elements must belong to different columns. When the first phase is vertical, the reasoning is the same.

Three phases

It turns out, that we can always sort the matrix in three phases. Furthermore, the type of the first phase is arbitrary, so let's say it's horizontal. After the first phase, we want the matrix to satisfy the *Two phases* condition, and more specifically, each column should consist of numbers that belong to different rows. Let's say, that the number has a color c if it belongs to row c. The algorithm will fix columns of the new matrix one by one. In each step, we want to permute the rows (but not the numbers from the fixed columns), so that the current column will have a number of every color. Actually, we can express this problem as a matching between rows and colors. We add an edge between a row and a color, if the row contains at least one non fixed numbers of that color. We want to argue, that we can always find a perfect matching.

Proof

Suppose we haven't fixed i columns yet. Then, there are exactly i numbers of every color left, and in each row there are i non-fixed items.
The existence of a perfect matching follows from Hall's theorem. When we take any subset of k rows, the set of non-fixed numbers in them is of size $k \cdot i$. Then since there are only i numbers of each color left, the numbers from this set are of at least k different colors.

We can implement it in time $\mathcal{O}(n^3\sqrt{n})$ using Hopcroft-Karp algorithm.

K: Shadow

You are given a point (source of light), sphere (death star) and plane (Earth). All of them are disjoint. Your goal is to compute area of shadow casted by death star on Earth.

First of all, we can move all the points, such that the sun is in $(0, 0, 0)$. Let's look at the half-lines starting in the origin and tangential to the sphere. They form an infinite cone. The shadow is the intersection of this cone and the plane, so it is a ellipse.

First of all, we can move all the points, such that the sun is in $(0, 0, 0)$. Let's look at the half-lines starting in the origin and tangential to the sphere. They form an infinite cone. The shadow is the intersection of this cone and the plane, so it is a ellipse.

Then we should find a plane which is going through the origin and the centre of the sphere and its perpendicular to the given plane (Earth). There will be exactly one such plane, except the case when this shadow is a circle, then we can take any such plane. The intersection of this plane and Earth is a line, which contains the major axis of the ellipse.

First of all, we can move all the points, such that the sun is in $(0, 0, 0)$. Let's look at the half-lines starting in the origin and tangential to the sphere. They form an infinite cone. The shadow is the intersection of this cone and the plane, so it is a ellipse.

Then we should find a plane which is going through the origin and the centre of the sphere and its perpendicular to the given plane (Earth). There will be exactly one such plane, except the case when this shadow is a circle, then we can take any such plane. The intersection of this plane and Earth is a line, which contains the major axis of the ellipse.

To find this major axis we should move into 2D problem on this new plane. The intersection of this plane and the sphere is a circle, so we have to find the half-lines starting in the origin and tangential to this circle. There will be two of them and they will determine the endpoints of the major line.

To find the minor axis, first we need to observe, that it is contained in the line lying in the Earth plane, perpendicular to the line containing the major axis. To find this minor axis endpoints we can move to the 2D problem as before or write some equations (using Pythagorean Theorem). After detecting the minor and major axis we can calculate the area of this ellipse with some common formulas.

J: Planet of the singles

You are given two binary string of length n. You can do following operations on first string:

- Change 0 to 1, it costs t_0
- Change 1 to 0, it costs t_1
- Swap two adjacent characters, it costs t_s.

Your goal is to change first string to second with minimum cost.

Observation 1

The given set of operations is equivalent to the following one:

1. change 0 to 1 on some fixed position in t_0 seconds,
2. change 1 to 0 on some fixed position in t_1 seconds,
3. swap two different bits on <u>any</u> two fixed positions i, j in $|i - j| t_s$ seconds.

Observation 1

The given set of operations is equivalent to the following one:

1. change 0 to 1 on some fixed position in t_0 seconds,
2. change 1 to 0 on some fixed position in t_1 seconds,
3. swap two different bits on <u>any</u> two fixed positions i, j in $|i - j| t_s$ seconds.

Observation 2

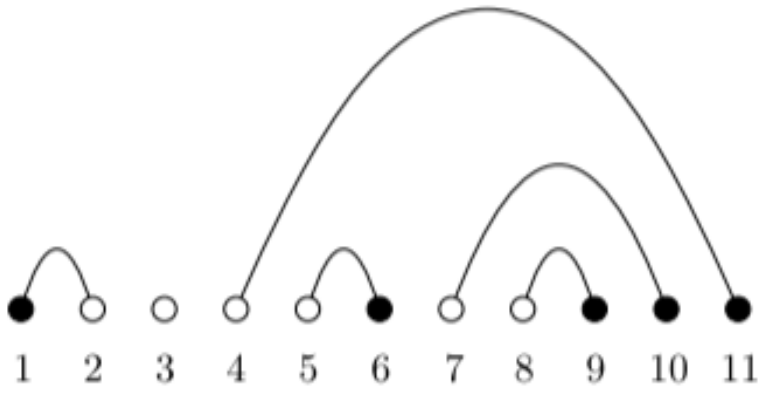
Let's call the binary strings from the input as A and B. There exists an optimal solution, which is not using any of the positions i, such that $A(i) = B(i)$. So we may only consider the positions such that $A(i) \neq B(i)$. Implementation note: if we decide to erase this unnecessary positions, we have to remember the initial positions of each pair, which is not deleted. Now (after erasing unnecessary pairs of positions), we can code our two binary strings into one binary string C in the following way. $C(i) = 0$ if $A(i) = 0 \wedge B(i) = 1$ and $C(i) = 1$ if $A(i) = 1 \wedge B(i) = 0$.

Let's try to understand our problem, looking at the string C. We can left some of the positions and pay respectively t_0 or t_1 seconds for each of them and the rest of them should be paired and for each such pair we should pay $t_s|\text{init}(i) - \text{init}(j)|$, where $\text{init}(i)$ is the position of this pair of bits in the initial strings.

Let's try to understand our problem, looking at the string C. We can left some of the positions and pay respectively t_0 or t_1 seconds for each of them and the rest of them should be paired and for each such pair we should pay $t_s|\text{init}(i) - \text{init}(j)|$, where $\text{init}(i)$ is the position of this pair of bits in the initial strings.

Observation 3

There exists an optimal solution in which this swaps "do not cross", i.e. for every two swaps (i_1, j_1) and (i_2, j_2) $(i_1 < i_2)$: $i_1 < j_1 < i_2 < j_2$ or $i_1 < i_2 < j_2 < j_1$.



Observation 4

There exists an optimal solution in which the following holds: if we use a swap for some pair (i, j), then all indices k such that $i < k < j$ are also used in some swaps (as in the previous picture).

Now, this problem can be solved using stack and some basic dynamic programming. First, we should calculate `left[i]` – the largest position j such that we can make swap (j, i) and satisfy the rule above. Then we should calculate `dp[i]` – the answer for the positions $\{1, 2, \ldots, i\}$. To calculate `dp[i]` we need to check two scenarios: we can skip i-th position and pay t_0 or t_1 depending on the bit or we can make a swap and use the value `left[i]` to calculate the answer for this case. The complexity is $\mathcal{O}(n)$.

A: Boys don't cry

We define operation f, which takes permutation π of length m and returns arbitrary permutation of length m that can be created with following algorithm:

- Start with empty permutation
- Analyze elements in order $\pi_1, \pi_2, \ldots, \pi_m$.
- Element π_i can be appended either to the beginning or to the end of current permutation.

You are given list of n permutations $\pi_1, \pi_2, \ldots, \pi_n$ of length m. We call permutation σ good if it can be achieved from all permutations given in the input with single application of operation f. Find number of good permutations and the smallest lexicographically good permutation.

Assume that there exist at least one good permutation. Our algorithm will work with that assumption, construct the smallest lexicographically permutation and then check if it is good.

Lemma 1

If σ is a good permutation then either $\sigma_1 = \pi_{1,m}$ or $\sigma_m = \pi_{1,m}$.

Assume that there exist at least one good permutation. Our algorithm will work with that assumption, construct the smallest lexicographically permutation and then check if it is good.

Lemma 1

If σ is a good permutation then either $\sigma_1 = \pi_{1,m}$ or $\sigma_m = \pi_{1,m}$.

Proof

When we apply f on π_1, algorithm inside f can append $\pi_{1,m}$ only to the beginning or to the end.

From Lemma 1 we know that $\pi_{1,m}$ is lying on one of the ends of every good permutation. If for all $k \in [1, n]$, $\pi_{k,m} = \pi_{1,m}$ then we can recursively solve our problem for permutation of length $m - 1$ and at the end append $\pi_{1,m}$ to the end/beginning in order to achieve smallest lexicographically permutation. Moreover number of good permutations is multipled by 2.

From Lemma 1 we know that $\pi_{1,m}$ is lying on one of the ends of every good permutation. If for all $k \in [1, n]$, $\pi_{k,m} = \pi_{1,m}$ then we can recursively solve our problem for permutation of length $m - 1$ and at the end append $\pi_{1,m}$ to the end/beginning in order to achieve smallest lexicographically permutation. Moreover number of good permutations is multipled by 2.

Otherwise we know that we can find v such that $\pi_{v,m} \neq \pi_{1,m}$. We can deduce that $\pi_{v,m}$ and $\pi_{1,m}$ lie on different ends of good permutation. If for all $k \in [1, n]$, $\pi_{k,m-1}, \pi_{k,m} \in \{\pi_{1,m}, \pi_{v,m}\}$ then again we can solve it similarly as above, but this time we solve for permutation of length $m - 2$.

But what if above statement is false? Then we can deduce for all elements in last 2 columns if those should go to end with $\pi_{1,m}$ or with $\pi_{v,m}$. We will continue this process until we reach the first moment when all permutations have the same set of elements in last C columns and solve recursively a smaller subproblem.

Complexity of algorithm is $\mathcal{O}((n+m)\cdot m)$, it can be optimized to $\mathcal{O}(n\cdot m)$, but either of them should be accepted.

D: Kiler

You are given an integer sequence $a_1, a_2, \ldots, a_n$ and q queries. Single query is of the form $a_p = v$. Your task is to print $\min(s, k)$ after each query, where s denotes length of longest increasing subsequence and k is a small constant given in the input.

Let's assume for a while that after each query $s \leq k$.

Definition

Let's represent each element a_i as a point (i, a_i) on a plane. We call point Q a predecessor of point P if $Q_x < P_x$ and $Q_y < P_y$.

Let's assume for a while that after each query $s \leq k$.

Definition

Let's represent each element a_i as a point (i, a_i) on a plane. We call point Q a predecessor of point P if $Q_x < P_x$ and $Q_y < P_y$.

Definition

Let T_r be a set of points P which:

- have a predecessor in T_{r-1} and **don't** have predecessor in T_r if $r \geq 2$.
- have no predecessor if $r = 1$

T_r is essentially a set of points for which the length of the LIS ending within the corresponding element in sequence a is equal to r, so our result is largest r for which T_r is nonempty.

Lemma

If we sort points in T_r by first coordinate so $x_1 < x_2 < \ldots < x_m$ then $y_1 \geq y_2 \geq \ldots \geq y_m$.

Lemma

If we sort points in T_r by first coordinate so $x_1 < x_2 < \ldots < x_m$ then $y_1 \geq y_2 \geq \ldots \geq y_m$.

Proof

If there exist a pair i, j such that $y_i < y_j$ then point j has a predecessor in T_r which contradicts definition of T_r.

Lemma

If we sort points in T_r by first coordinate so $x_1 < x_2 < \ldots < x_m$ then $y_1 \geq y_2 \geq \ldots \geq y_m$.

Proof

If there exist a pair i, j such that $y_i < y_j$ then point j has a predecessor in T_r which contradicts definition of T_r.

Let's keep sets $T_1, T_2, \ldots, T_k$ as BSTs with split/merge (for example treaps or AVLs if you prefer deterministic BST).
Quries can be seen as removing point (p, a_p), setting $a_p := v$ and adding point (p, a_p). So our goal is to efficiently modify $T_1, T_2, \ldots T_k$ after each insertion/deletion.

Insertion

First we can easily find the set T_r where our point P belongs – it is the first set where we can't find P's predecessor. It can be done in $\mathcal{O}(k \log n)$ or $\mathcal{O}(\log n \log k)$.

After locating right T_r we might need to remove few elements from T_r for which P is a predecessor and add them to T_{r+1}. Because there can be many such points we can't iterate over all those points and recursively add them to T_{r+1}.

Luckily it turns out that those points form a contiguos interval on T_r which can be easily find in $\mathcal{O}(\log n)$ with binsearch on T_r. But now we have to add this interval to T_{r+1}.

Surprisingly we can do the same procedure as we did with one point! So we keep interval of points to add, remove points from current set which points from interval precede, add this interval and recursively add deleted points to the next set.

With merge and split we can perform insert in $\mathcal{O}(k \log n)$.

Removal

It turns out that removing elements is quite similar to insertion. When we need to remove point P from T_r we also need to cut points from T_{r+1} which are preceded by P. Those points form interval so we can easily cut them from T_{r+1} and insert to T_r. Then we need to cut points from T_{r+2}, because some of them might lost predecessor. We can do it in the same manner as we did for T_{r+1} and continue whole process recursively. Complexity for removal is $\mathcal{O}(k \log n)$.

But how to solve this problem when we can't assume that legth of LIS is $\leq k$? It turns out that's quite easy – we can do lazy insertion!

But how to solve this problem when we can't assume that legth of LIS is $\leq k$? It turns out that's quite easy – we can do lazy insertion!
We will keep a set of points S which are **not** added to sets $T_1, T_2, \ldots, T_k$. So when we insert a point and T_k is non empty then we can just add it to S and forget about it for a while.
If at some point set T_k is empty then we will keep adding points from S until set T_k becomes nonempty or we run out of points. This guarantees us that we can calculate s precisely if it is smaller then k and that adding/removing a point will take at most k recursive insertions/removals, so we obtain complexity $\mathcal{O}(n \log n + q \cdot k \log n)$.

I: Interest

Given a directed graph of n vertices and m edges in which each vertex is reachable from vertex 1, for each vertex other than 1, find the minimal sum of lengths of edge disjoint paths from vertex 1 to this vertex, or determine that it is impossible to find such pair of paths.

First approach

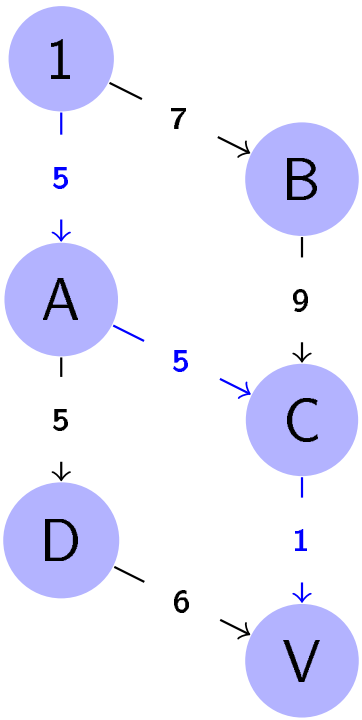
For a moment let's focus on getting the answer for just one vertex V. One can see that we can solve this problem using two iterations of Minimum Cost Maximum Flow algorithm. Next few slides will try to provide short explanation of this process.

We find the minimal length path from 1 to V using Dijkstra. Next we want to reverse the direction of edges we used and make their cost negative. If we then find the shortest path, we will obtain the answer.

First approach

For a moment let's focus on getting the answer for just one vertex V. One can see that we can solve this problem using two iterations of Minimum Cost Maximum Flow algorithm. Next few slides will try to provide short explanation of this process.

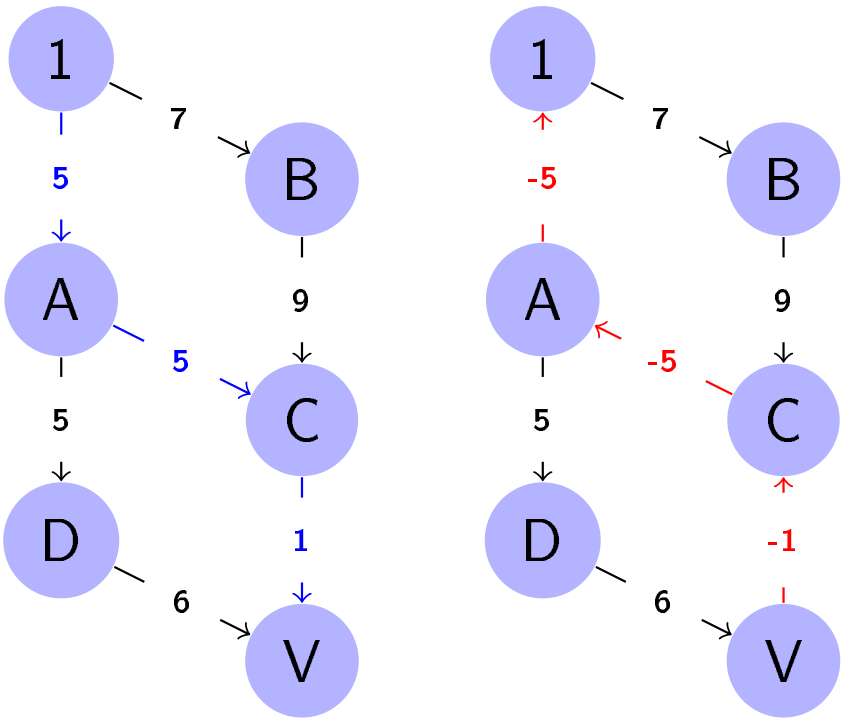
We find the minimal length path from 1 to V using Dijkstra. Next we want to reverse the direction of edges we used and make their cost negative. If we then find the shortest path, we will obtain the answer.



First approach

For a moment let's focus on getting the answer for just one vertex V. One can see that we can solve this problem using two iterations of Minimum Cost Maximum Flow algorithm. Next few slides will try to provide short explanation of this process.

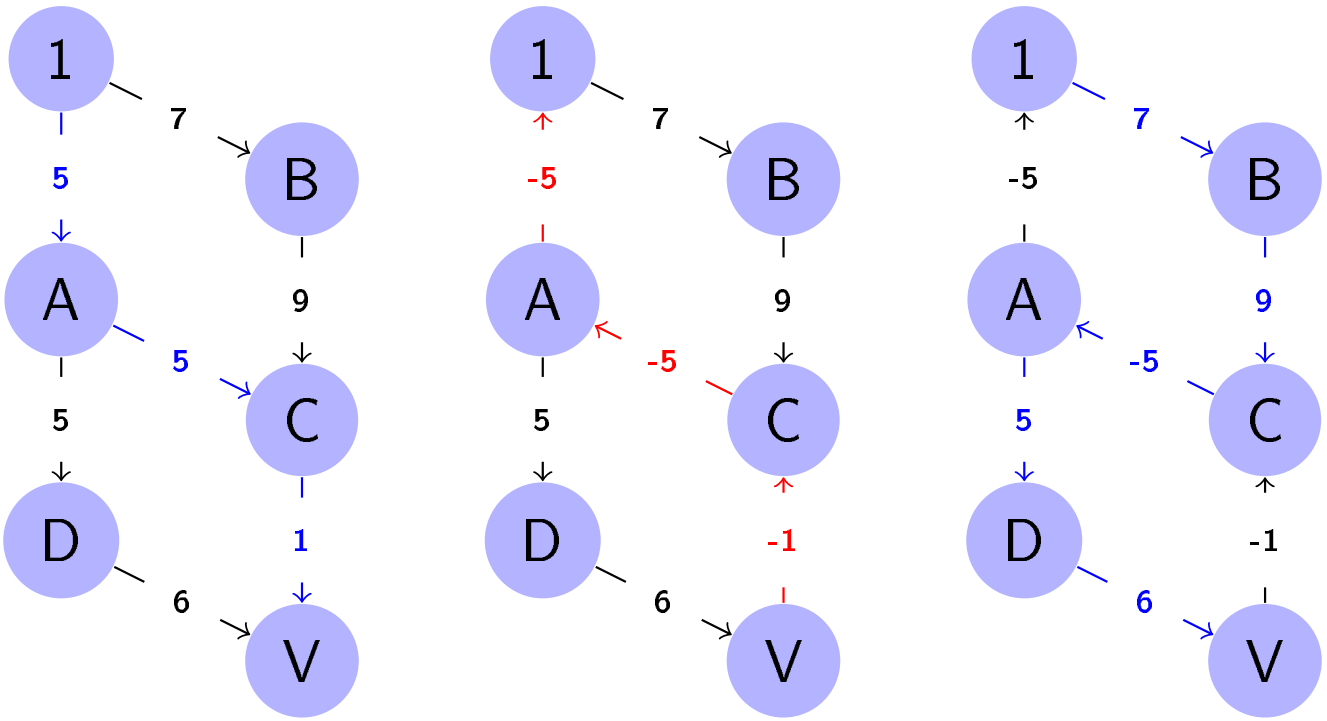
We find the minimal length path from 1 to V using Dijkstra. Next we want to reverse the direction of edges we used and make their cost negative. If we then find the shortest path, we will obtain the answer.



First approach

For a moment let's focus on getting the answer for just one vertex V. One can see that we can solve this problem using two iterations of Minimum Cost Maximum Flow algorithm. Next few slides will try to provide short explanation of this process.

We find the minimal length path from 1 to V using Dijkstra. Next we want to reverse the direction of edges we used and make their cost negative. If we then find the shortest path, we will obtain the answer.



Issue

After the first phase we introduced negative weight edges, so we no longer can just run Dijkstra algorithm. It is worth observing that adding those negative weight edges will not result in creation of negative cost cycles (if such cycle occurred, then the path could not be the shortest one).

Issue

After the first phase we introduced negative weight edges, so we no longer can just run Dijkstra algorithm. It is worth observing that adding those negative weight edges will not result in creation of negative cost cycles (if such cycle occurred, then the path could not be the shortest one).

Negative edges

Using the edge with negative weight in the second phase of algorithm would correspond to removing this edge from the contribution to the answer.

Issue

After the first phase we introduced negative weight edges, so we no longer can just run Dijkstra algorithm. It is worth observing that adding those negative weight edges will not result in creation of negative cost cycles (if such cycle occurred, then the path could not be the shortest one).

Negative edges

Using the edge with negative weight in the second phase of algorithm would correspond to removing this edge from the contribution to the answer.

In this example, as a solution we obtained two edge-disjoint paths:

$$1 \to B \to C \to V \quad \text{and} \quad 1 \to A \to D \to V$$

Which are indeed optimal (and only) pair of disjoint-paths from 1 to V.

We have to find a way of getting rid of these negative edges. To do so we will have to introduce so called *function of potential* $p(x)$ (like in Johnson algorithm), which in our case, for each vertex will return the length of the shortest path in the original graph from 1 leading to this vertex.

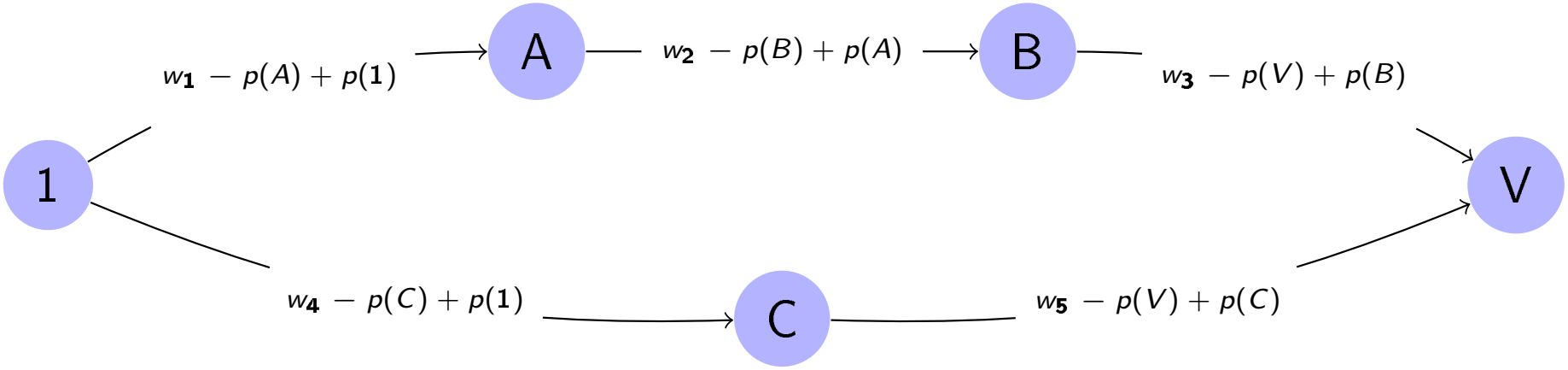
We have to find a way of getting rid of these negative edges. To do so we will have to introduce so called *function of potential* $p(x)$ (like in Johnson algorithm), which in our case, for each vertex will return the length of the shortest path in the original graph from 1 leading to this vertex.

Modification

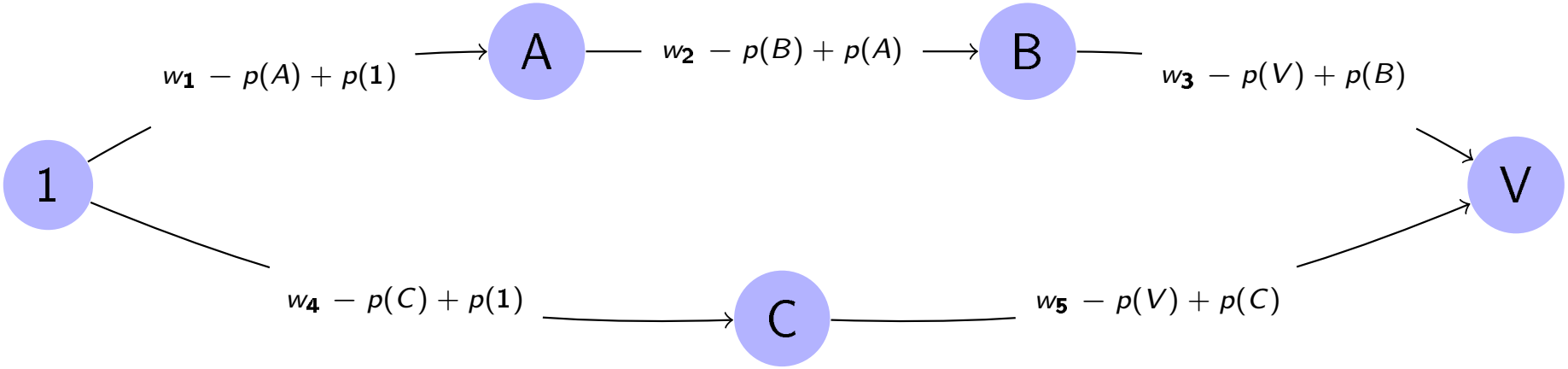
For each edge $u \to v$ with cost w we will change its weight w to $c(w) = w - p(v) + p(u)$. No edge will have negative weight, because $p(v) \geq p(u) + w$. Note that edges on the Dijkstra tree will have cost equal to 0.

The key observation is that if we consider all the paths from the vertex 1 to the vertex V, lengths of all of them will be decreased by the same constant value $p(V)$. Knowing this, if we find the shortest path S to vertex V in modified graph and then increase it twice by the value of $p(V)$ we will obtain the answer for this vertex!

The key observation is that if we consider all the paths from the vertex 1 to the vertex V, lengths of all of them will be decreased by the same constant value $p(V)$. Knowing this, if we find the shortest path S to vertex V in modified graph and then increase it twice by the value of $p(V)$ we will obtain the answer for this vertex!



The key observation is that if we consider all the paths from the vertex 1 to the vertex V, lengths of all of them will be decreased by the same constant value $p(V)$. Knowing this, if we find the shortest path S to vertex V in modified graph and then increase it twice by the value of $p(V)$ we will obtain the answer for this vertex!



Complexity

Using this method we obtained the solution which runs in $\mathcal{O}(n(n+m)\log m)$, which is definitely too slow to pass the tests.

To pass the tests, we will try to compute the answer for all vertices simultaneously. We'd like to:

1. Run Dijkstra algorithm and find arbitrary Dijkstra tree (let's call it D).
2. Flip the direction and change the cost to negative of edges in D.
3. Modify all edges in whole graph using potential function.
4. Find the shortest path in the modified graph (M) to each vertex.

To pass the tests, we will try to compute the answer for all vertices simultaneously. We'd like to:

1. Run Dijkstra algorithm and find arbitrary Dijkstra tree (let's call it D).
2. Flip the direction and change the cost to negative of edges in D.
3. Modify all edges in whole graph using potential function.
4. Find the shortest path in the modified graph (M) to each vertex.

Issue

If we now look at some vertex v, it could happen that we flipped some edges that were not on the shortest path from 1 to this vertex. This prevents us from using classic Dijkstra algorithm in the second phase.

This is where the magic begins! We will now show the method of modifying Dijkstra algorithm so that is still provides us with good result for each of the vertexes.

This is where the magic begins! We will now show the method of modifying Dijkstra algorithm so that is still provides us with good result for each of the vertexes.

After first phase and reversing edges in D, we will assume that all vertexes lie in the same connected component. Two vertices are in the same component if there still exist a path from one to the other in D (if we ignore directions or edges). Now, similarly like in Dijkstra algorithm we will consider a vertex closest to the vertex 1 in M (let's call him u). We state that the shortest path to u cannot possibly change in the future due to the fact that edges are non negative. In the next step, we would like to relax all the edges coming out of this vertex.

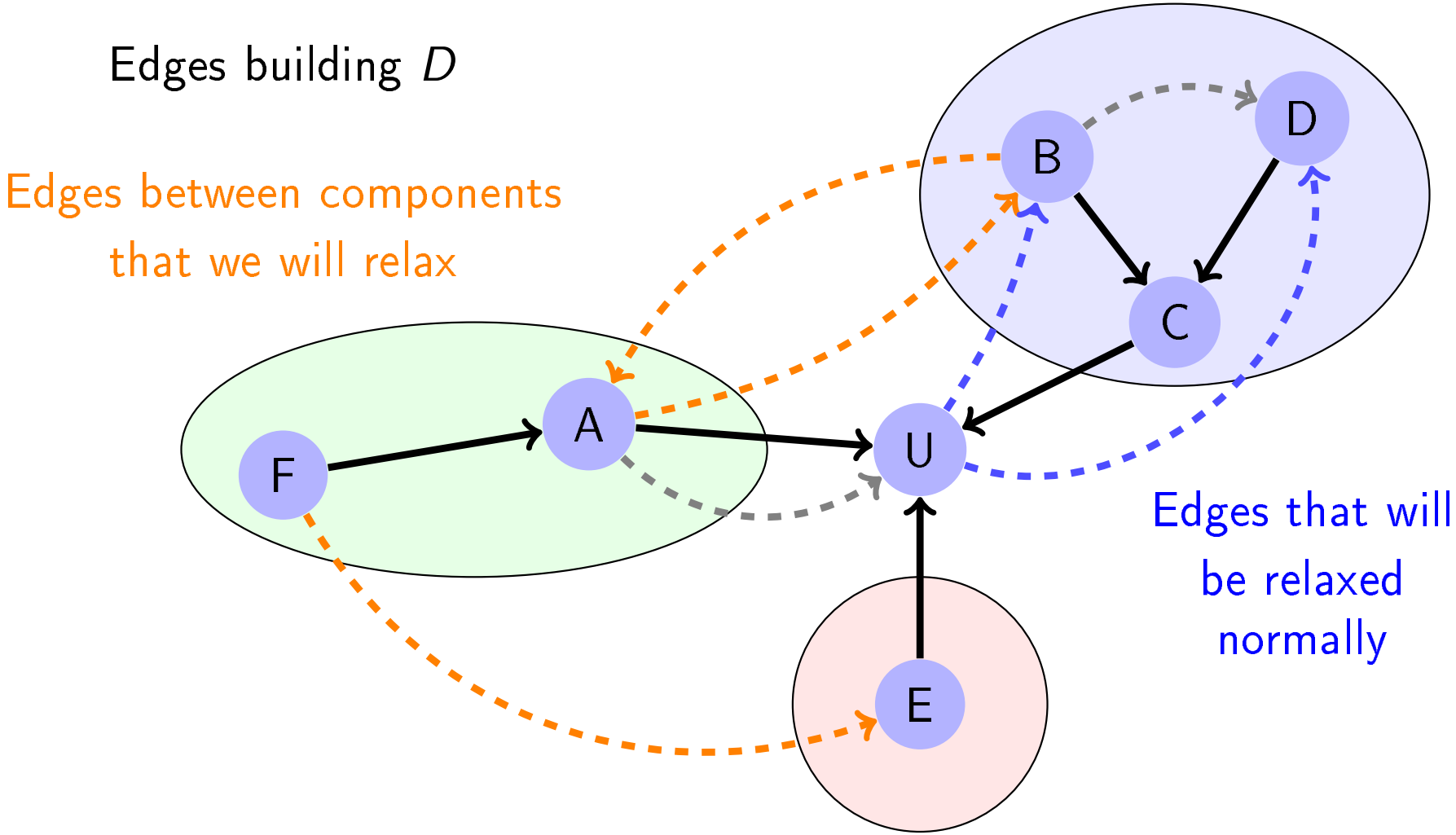
Relaxation

At this point we assume, that we relaxing vertex u. Let's consider all edges coming out of u.

- Edges not contained in D (dotted line). Those edges will be relaxed almost the same way like in Dijkstra's algorithm. We only have to check if the edge lead's us to one of newly emerged components.
- Edges in D. This is where it gets tricky. Removal of vertex u may create some new connected components. We want to relax all the edges that do not lie in D and connect two different newly emerged components. For each such edge $A \to B$, we relax vertex B with the cost of reaching vertex u increased by the cost of this edge. Why would it work? Well... the prof is based on case analysis, but in brief, this corresponds to reaching vertex A using edges in D (which will have cost 0).

Ater relaxation we remove all the edges incident to vertex u.

Edges not used during that phase



Example above shows only the component including U.

Implementation

Now we have to find a way to somehow implement it efficiently. Each edge in our graph will be relaxed at most once. But if we just iterate all edges in corresponding connected components, we will run into trouble, because at each phase of relaxing, there may be a very big component, which we will traverse, and we won't use most of it's edges. Fortunately for us if we only iterate over the components that are not the greatest, we will traverse each edge in worst case $\mathcal{O}(\log N)$ times, because when we traverse the edge the size of component it is in gets decreased at least twice.

Implementation

Now we have to find a way to somehow implement it efficiently. Each edge in our graph will be relaxed at most once. But if we just iterate all edges in corresponding connected components, we will run into trouble, because at each phase of relaxing, there may be a very big component, which we will traverse, and we won't use most of it's edges. Fortunately for us if we only iterate over the components that are not the greatest, we will traverse each edge in worst case $\mathcal{O}(\log N)$ times, because when we traverse the edge the size of component it is in gets decreased at least twice.

How to check if the component is the greatest one?

You can choose two arbitrary components and iterate them simultaneously. Now we will check both edges going *in* and *out* of the vertexes. If you run out of edges in one of them, then you stop iterating the other and choose another pair of components, until there is only one left.

Complexity: $\mathcal{O}((m+n)\log m)$